THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

DATA 37100 Intro to AI: DL and GAI

2026 Winter

Diffusion Model

Edwin Lo

Where we stand, and next step

- We built a toolkit of generative models:
  AE → VAE → GAN → Transformers/LLMs
- This week: diffusion = learn to remove noise step-by-step
- Learn about:
  - ✓ DDPM: Denoising Diffusion Probabilistic Models – probabilistic (stochastic) reverse process; typically requires many sampling steps. (Ho, Jain, Abbeel, 2020)
  - ✓ (little bit of) DDIM: Denoising Diffusion Implicit Models – Deterministic / Implicit sampling formulation derived from the same training objective; enables faster sampling with fewer steps. (Song, Meng, Ermon, 2020)

Add Noise (easy)

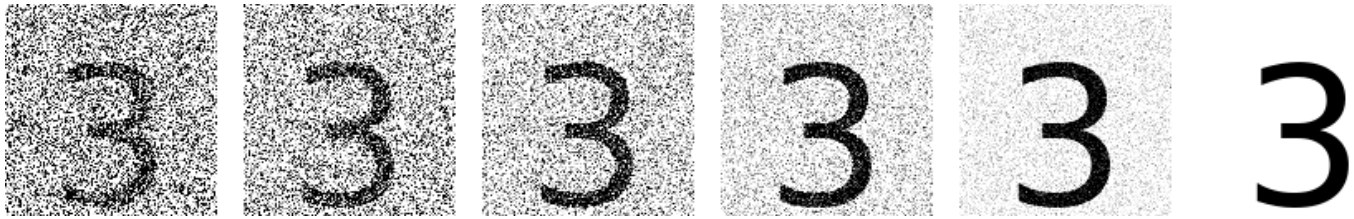
Forward process (known): add a little noise each step

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Add Noise (easy) and Denoise (learned)

Forward process (known): add a little noise each step

Reverse process (learned): remove noise each step



Key trick: train a neural net $\epsilon_\theta(x_t, t)$ to predict the injected noise ϵ. Then sampling is iterative denoising from pure noise.

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Forward diffusion: a simple Markov chain

Define $\beta_t \in (0,1)$ (a "noise schedule"). Then:

$$q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t} \cdot x_{t-1}, \beta_t I)$$

"Tiny Gaussian corruption" repeated T times $\Rightarrow x_T \approx \mathcal{N}(0, I)$

- ✓ Forward process is fixed (no learning).
- ✓ The model only needs to learn the reverse transitions.
- ✓ With the right objective, that learning becomes surprisingly simple.

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Sample x_t from x_0 in one shot

Let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$. Then:

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

Reparameterization means training can pick a random timestep t, construct x_t directly from x_0, and ask the network to recover ϵ.

```
# training step (simplified)
x0 ~ data
t ~ Uniform({1..T})
eps ~ N(0, I)
xt = sqrt(alpha_bar[t])*x0 + sqrt(1-alpha_bar[t])*eps
loss = || eps - eps_theta(xt, t) ||^2
```

Reverse: Gaussian w/ a learned mean

We model the reverse transition as:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(\mu_\theta(x_t|t), \sigma_t^2 I)$$

Instead of predicting μ_θ directly, DDPM predicts the noise $\epsilon_\theta(x_t, t)$. That gives a clean formula for μ_θ and stabilize training.

$$\mu_\theta(x_t|t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \left(\frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\right) \cdot \epsilon_\theta(x_t, t)\right)$$

At sampling time: start from $x_T \sim \mathcal{N}(0, I)$ and iterate $t = T \ldots 1$.

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Training: “Predict the noise” (simple MSE)

Simplified objective (common in practice):

$$\mathcal{L} = \mathbb{E}_{x_0 \sim q(x_0), \epsilon \sim N(0,I), t \sim \mathrm{Uniform}(1,T)}[\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

Interpretation:

- ✓ At timestep t, x_t is a noisy version of x_0.
- ✓ The network learns a denoiser conditioned on “noise level” t.
- ✓ This connects diffusion to denoising score matching / score-based models.

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Why Predicting ϵ Simplifies the ELBO

- ✓ Original objective: maximize variational lower bound on $\log p(x_0)$
- ✓ KL terms appear between Gaussian forward & reverse transitions
- ✓ With fixed variance choice, objective reduces to weighted MSE
- ✓ Training loss becomes:

$$\mathcal{L} = \mathbb{E}\,\|\epsilon - \epsilon_\theta(x_t, t)\|^2$$

- ✓ Key Insight: Predicts ϵ transforms variational inference into simple regression.

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Architecture: U-Net conditioned on time

Most image diffusion models use a U-Net backbone:

Conditioning: embed timestep t (and optionally class/text) and inject into blocks.

Output: $\epsilon_\theta(x_t, t)$ (same shape as image).

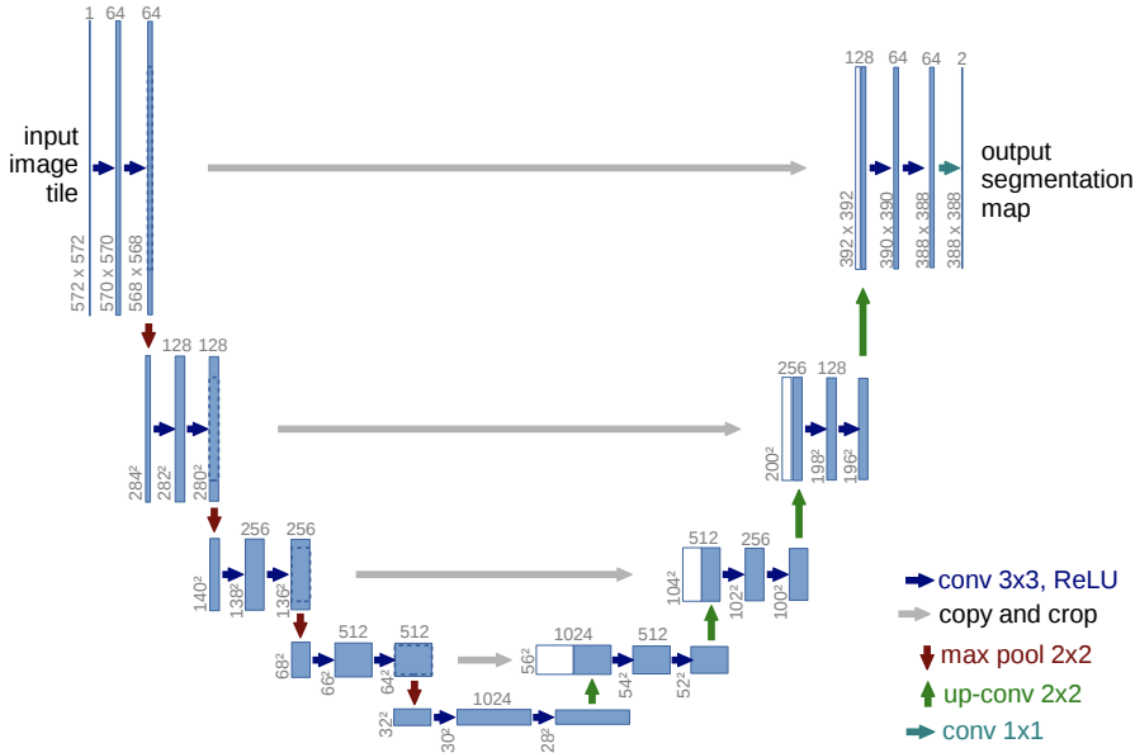
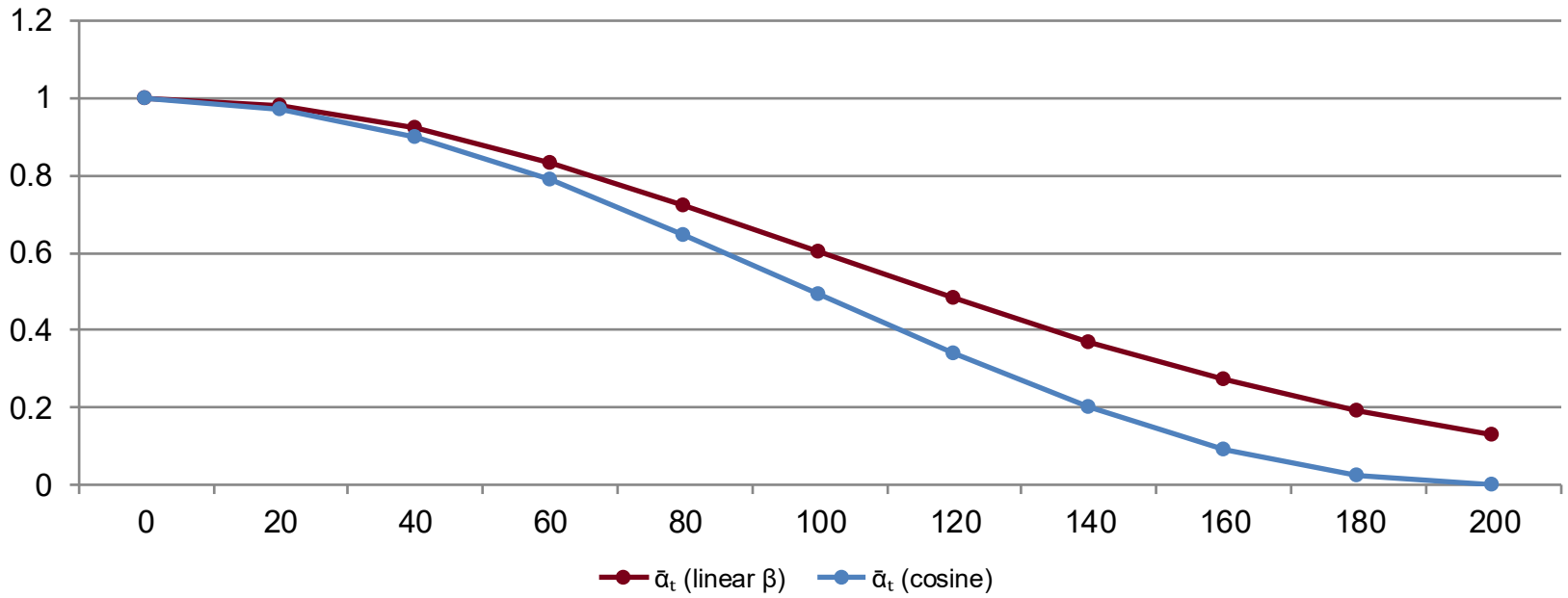


Fig. 1. U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

https://nn.labml.ai/diffusion/ddpm/unet.html

Noise schedule matters

Example: cumulative signal $\bar{\alpha}_t$ under two schedules (T=200)



Rule of thumb: you want a smooth progression where early steps are learnable and late steps reach near-pure noise.
Cosine schedules are a common improvement over the original linear schedule.

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Signal-to-Noise Ratio (SNR) Perspective

- Define cumulative signal: $\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$
- Signal-to-Noise Ratio: $\mathrm{SNR}_t = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}$
- Interpretation:
  - ✓ Early timesteps → high SNR → structure preserved
  - ✓ Late timesteps → low SNR → near pure noise
- Schedule design shapes SNR decay curve
- Cosine schedule yields smoother degradation.

Sampling: DDPM vs DDIM

- DDPM sampling (ancestral):
  - ✓ iterate $t = T \ldots 1$
  - ✓ add a little randomness each step (stochastic)
  - ✓ high quality but often needs many steps

- DDIM (implicit):
  - ✓ same training objective
  - ✓ deterministic (or partially stochastic) update
  - ✓ can sample in fewer steps by skipping timesteps

Practical takeaway: training is "slow once", sampling is "fast many". DDIM (and later samplers) focus on making sampling faster.

Conditional diffusion & CFG

- To generate specific outputs, condition the model on c (class label, text prompt, …).
- Classifier-free guidance (CFG) mixes two predictions:
- $\hat{\epsilon} = (1 + w) \cdot \epsilon_\theta(x_t, t, c) - w \cdot \epsilon_\theta(x_t, t, \phi)$
- w controls the tradeoff: fidelity ↑ vs diversity ↓

In this week's first lab we do unconditional MNIST (no c).
But the same mechanics power text-to-image systems (with a text encoder + attention).

lab07_diffusion

- lab07_diffusion_core.py (train + sample)
  - ✓ Terminal:
    python lab07_diffusion_core.py --dataset mnist --epochs 5 --T 200 --bs 128 --lr 2e-4

- lab07_diffusion_ui.py (widgets UI)

- Watch for
  - ✓ samples/ — output images during training (quality sanity-check)
  - ✓ metrics.jsonl — loss trend (should generally decrease)
  - ✓ config.json — exact specs for the run (reproducibility)

Potential Issues (and fixes)

- ✓ Outputs look like pure noise for a long time → train longer / reduce LR / increase model width
- ✓ Samples "collapse" (all similar) → try different schedule / seed / more data augmentation
- ✓ Training is slow → reduce T for class demo; later discuss DDIM / fewer steps
- ✓ macOS multiprocessing error → run with --num-workers 0 (we default to this)
- ✓ MPS/CPU differences → keep batch size modest; avoid too many workers

Takeaways

- ✓ Outputs Diffusion models learn to invert a gradual noising process.
- ✓ Training reduces to predicting noise ε with a timestep-conditioned U-Net.
- ✓ Sampling is iterative denoising; faster samplers aim to reduce steps.
- ✓ Next week (wk8): evaluation, ethics, and societal impact.

References

- Sohl-Dickstein et al. (2015): https://arxiv.org/abs/1503.03585
- Ho, Jain, Abbeel (2020) DDPM: https://arxiv.org/abs/2006.11239
- Song, Meng, Ermon (2020) DDIM: https://arxiv.org/abs/2010.02502
- Song et al. (2021) score-based SDEs: https://arxiv.org/abs/2011.13456
- Nichol & Dhariwal (2021): https://arxiv.org/abs/2102.09672
- Ho & Salimans (2022) CFG: https://arxiv.org/abs/2207.12598

Bonus Question

Why didn't diffusion dominate before GANs?

1. Sampling cost
   - ✓ Diffusion requires T sequential denoising steps
   - ✓ GANs generate in one forward pass
2. Early engineering limitations
   - ✓ Less effective schedules
   - ✓ No ϵ-parameterization insight
   - ✓ Smaller models and datasets
3. Hardware & scale shift
   - ✓ Better GPUs and large-batch training
   - ✓ Improved samplers (e.g., DDIM)
4. Takeaway:
   - ✓ GANs were faster but fragile; diffusion is slower but stable.
   - ✓ When compute increased, stability won.

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE